

GenAI Code Comparison Analysis

EECS 348 - Software Engineering I - Assignment 2

Charvi Reddy Konudula

a. GenAI Programs Used

Two GenAI tools were used to generate an initial C implementation: **ChatGPT** and **Google Gemini**.

b. How Each Was Accessed

Both tools were accessed through their web interfaces in the Google Chrome browser: ChatGPT was accessed at chat.openai.com, and Gemini was accessed at gemini.google.com. No desktop apps, extensions, or API access were used - both were plain browser chat sessions.

c. Prompt Used (identical for both GenAIs)

Write a C program that implements a priority queue for a CEO's email inbox using a MaxHeap.

Requirements:

- Implement the MaxHeap yourself using a list-based (array-based) implementation. Do not use any pre-existing/library heap modules - all heap functions (insert, extract-max, heapify, etc.) must be written from scratch.
- Use the MaxHeap to store and prioritize emails.

Priority order (highest to lowest):

1. Boss
2. Subordinate
3. Peer
4. ImportantPerson
5. OtherPerson

If multiple emails have the same sender category, the newest email (by date) should have higher priority than older ones.

The program reads commands from a text file, one per line:

- EMAIL <sender category>,<subject line>,<date>
Adds an email to the inbox. <sender category> is one of the five strings above. <subject line> is a string that may contain spaces but not commas. <date> is in MM-DD-YYYY format.
- NEXT
Displays the next (highest priority) email in this format:
Sender:
Subject:
Date:
Does not remove it from the queue.
- READ
Removes the highest priority email from the queue without displaying it.
- COUNT
Displays the current count of unread emails in the format:
"There are N emails to read."

Handle edge cases:

- NEXT or READ when there are no emails in the queue.
- Two NEXTs in a row without an intervening READ (should show the same email both times).
- Two READs in a row without an intervening NEXT (second READ removes the next highest priority email without displaying it).

The program should read all commands from a file and process them in order. Provide the complete, compilable C source code.

d. Generated Code (Raw, Unedited Output)

d.1 - Gemini's Raw Output

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_CAPACITY 16

// Enum mapping categories to priority values (Higher number = Higher priority)
typedef enum {
    OTHER_PERSON = 1,
    IMPORTANT_PERSON = 2,
    PEER = 3,
    SUBORDINATE = 4,
    BOSS = 5
} PriorityCategory;

typedef struct {
    PriorityCategory category;
    char sender_str[32];
    char subject[256];
    char date_str[11]; // MM-DD-YYYY\0
    int year;
    int month;
    int day;
} Email;

typedef struct {
    Email *data;
    int size;
    int capacity;
} MaxHeap;

// Helper to convert string category to Enum
PriorityCategory parse_category(const char *cat) {
    if (strcmp(cat, "Boss") == 0) return BOSS;
    if (strcmp(cat, "Subordinate") == 0) return SUBORDINATE;
    if (strcmp(cat, "Peer") == 0) return PEER;
    if (strcmp(cat, "ImportantPerson") == 0) return IMPORTANT_PERSON;
    return OTHER_PERSON; // Default / OtherPerson
}

// Parses MM-DD-YYYY into integer components for easy comparison
void parse_date(const char *date_str, int *m, int *d, int *y) {
    sscanf(date_str, "%d-%d-%d", m, d, y);
}

/*
 * Priority Comparison:
 * Returns > 0 if email A has strictly HIGHER priority than email B.
 * Returns < 0 if email A has LOWER priority than email B.
 * Returns 0 if both are identical.
 */
int compare_emails(const Email *a, const Email *b) {
    // 1. Primary Priority: Sender Category
    if (a->category != b->category) {
        return a->category - b->category;
    }

    // 2. Secondary Priority: Date (Newer date = Higher priority)
    if (a->year != b->year) return a->year - b->year;
    if (a->month != b->month) return a->month - b->month;
    return a->day - b->day;
}

// Memory Allocation & Heap Lifecycle
MaxHeap* create_heap() {
    MaxHeap *heap = (MaxHeap*)malloc(sizeof(MaxHeap));
    heap->capacity = INITIAL_CAPACITY;
    heap->size = 0;
    heap->data = (Email*)malloc(heap->capacity * sizeof(Email));
    return heap;
}

void free_heap(MaxHeap *heap) {
    if (heap) {
        free(heap->data);
        free(heap);
    }
}
```

```

    }
}

void swap(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify up (Sift-Up) for Insertion
void heapify_up(MaxHeap *heap, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (compare_emails(&heap->data[index], &heap->data[parent]) > 0) {
            swap(&heap->data[index], &heap->data[parent]);
            index = parent;
        } else {
            break;
        }
    }
}

// Heapify down (Sift-Down) for Extraction
void heapify_down(MaxHeap *heap, int index) {
    int largest = index;
    int left = 2 * index + 1;
    int right = 2 * index + 2;

    if (left < heap->size && compare_emails(&heap->data[left], &heap->data[largest]) > 0) {
        largest = left;
    }

    if (right < heap->size && compare_emails(&heap->data[right], &heap->data[largest]) > 0) {
        largest = right;
    }

    if (largest != index) {
        swap(&heap->data[index], &heap->data[largest]);
        heapify_down(heap, largest);
    }
}

// Priority Queue Operations
void heap_insert(MaxHeap *heap, Email email) {
    // Dynamic resizing
    if (heap->size == heap->capacity) {
        heap->capacity *= 2;
        heap->data = (Email*)realloc(heap->data, heap->capacity * sizeof(Email));
    }

    heap->data[heap->size] = email;
    heapify_up(heap, heap->size);
    heap->size++;
}

// Removes and returns highest-priority email
int heap_extract_max(MaxHeap *heap, Email *result) {
    if (heap->size <= 0) return 0;

    *result = heap->data[0];
    heap->data[0] = heap->data[heap->size - 1];
    heap->size--;

    if (heap->size > 0) {
        heapify_down(heap, 0);
    }

    return 1;
}

// Peeks at highest-priority email without removal
int heap_peek_max(const MaxHeap *heap, Email *result) {
    if (heap->size <= 0) return 0;
    *result = heap->data[0];
    return 1;
}

// Command Processing
void process_command(MaxHeap *heap, char *line) {
    // Trim trailing newlines/carriage returns
    line[strcspn(line, "\r\n")] = '\0';
    if (strlen(line) == 0) return;

```

```

if (strncmp(line, "EMAIL ", 6) == 0) {
    char *payload = line + 6;
    Email email;

    // Parse comma-separated fields: <sender category>,<subject line>,<date>
    char *cat_tok = strtok(payload, ",");
    char *subj_tok = strtok(NULL, ",");
    char *date_tok = strtok(NULL, ",");

    if (cat_tok && subj_tok && date_tok) {
        strncpy(email.sender_str, cat_tok, sizeof(email.sender_str) - 1);
        strncpy(email.subject, subj_tok, sizeof(email.subject) - 1);
        strncpy(email.date_str, date_tok, sizeof(email.date_str) - 1);

        email.sender_str[sizeof(email.sender_str) - 1] = '\0';
        email.subject[sizeof(email.subject) - 1] = '\0';
        email.date_str[sizeof(email.date_str) - 1] = '\0';

        email.category = parse_category(email.sender_str);
        parse_date(email.date_str, &email.month, &email.day, &email.year);

        heap_insert(heap, email);
    }
} else if (strcmp(line, "NEXT") == 0) {
    Email top;
    if (heap_peek_max(heap, &top)) {
        printf("Sender: %s\n", top.sender_str);
        printf("Subject: %s\n", top.subject);
        printf("Date: %s\n", top.date_str);
    } else {
        printf("There are no emails in the queue.\n");
    }
} else if (strcmp(line, "READ") == 0) {
    Email top;
    if (!heap_extract_max(heap, &top)) {
        printf("There are no emails to read.\n");
    }
} else if (strcmp(line, "COUNT") == 0) {
    printf("There are %d emails to read.\n", heap->size);
}
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <input_file.txt>\n", argv[0]);
        return 1;
    }

    FILE *file = fopen(argv[1], "r");
    if (!file) {
        perror("Error opening file");
        return 1;
    }

    MaxHeap *inbox = create_heap();
    char line[512];

    while (fgets(line, sizeof(line), file)) {
        process_command(inbox, line);
    }

    fclose(file);
    free_heap(inbox);
    return 0;
}

```

d.2 - ChatGPT's Raw Output

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_EMAILS 1000
#define MAX_SUBJECT 256
#define MAX_LINE 512

typedef struct {
    char sender[32];
    char subject[MAX_SUBJECT];
    char date[11]; /* MM-DD-YYYY + '\0' */
    int priority;
} Email;

typedef struct {
    Email data[MAX_EMAILS];
    int size;
} MaxHeap;

/* -----
   Priority helpers
   ----- */

int getPriority(const char *sender)
{
    if (strcmp(sender, "Boss") == 0)
        return 5;
    if (strcmp(sender, "Subordinate") == 0)
        return 4;
    if (strcmp(sender, "Peer") == 0)
        return 3;
    if (strcmp(sender, "ImportantPerson") == 0)
        return 2;
    if (strcmp(sender, "OtherPerson") == 0)
        return 1;

    return 0;
}

/*
 * Converts MM-DD-YYYY into YYYYMMDD.
 * Since the date format is fixed, comparing these integers
 * correctly compares chronological order.
 */
int dateValue(const char *date)
{
    int month, day, year;

    if (sscanf(date, "%d-%d-%d", &month, &day, &year) != 3)
        return 0;

    return year * 10000 + month * 100 + day;
}

/*
 * Returns nonzero if a should have higher priority than b.
 *
 * Sender category is compared first. If categories are equal,
 * the newer date gets higher priority.
 */
int higherPriority(const Email *a, const Email *b)
{
    if (a->priority != b->priority)
        return a->priority > b->priority;

    return dateValue(a->date) > dateValue(b->date);
}

/* -----
   MaxHeap implementation
   ----- */

void swapEmails(Email *a, Email *b)
{
    Email temp = *a;
    *a = *b;
    *b = temp;
}
```

```

void heapifyUp(MaxHeap *heap, int index)
{
    while (index > 0) {
        int parent = (index - 1) / 2;

        if (!higherPriority(&heap->data[index],
                           &heap->data[parent]))
            break;

        swapEmails(&heap->data[index], &heap->data[parent]);
        index = parent;
    }
}

void heapifyDown(MaxHeap *heap, int index)
{
    while (1) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        if (left < heap->size &&
            higherPriority(&heap->data[left],
                           &heap->data[largest])) {
            largest = left;
        }

        if (right < heap->size &&
            higherPriority(&heap->data[right],
                           &heap->data[largest])) {
            largest = right;
        }

        if (largest == index)
            break;

        swapEmails(&heap->data[index], &heap->data[largest]);
        index = largest;
    }
}

void initializeHeap(MaxHeap *heap)
{
    heap->size = 0;
}

int insert(MaxHeap *heap, Email email)
{
    if (heap->size >= MAX_EMAILS)
        return 0;

    heap->data[heap->size] = email;
    heapifyUp(heap, heap->size);
    heap->size++;

    return 1;
}

/*
 * Removes and returns the highest-priority email.
 * The caller can choose not to use the returned email, which
 * is what READ does.
 */
int extractMax(MaxHeap *heap, Email *result)
{
    if (heap->size == 0)
        return 0;

    if (result != NULL)
        *result = heap->data[0];

    heap->size--;

    if (heap->size > 0) {
        heap->data[0] = heap->data[heap->size];
        heapifyDown(heap, 0);
    }

    return 1;
}

/*

```

```

    * Returns the highest-priority email without removing it.
    * This is what NEXT needs.
    */
int peekMax(const MaxHeap *heap, Email *result)
{
    if (heap->size == 0)
        return 0;

    if (result != NULL)
        *result = heap->data[0];

    return 1;
}

/* -----
   Command processing
   ----- */

void processEmailCommand(MaxHeap *heap, char *line)
{
    char *firstComma;
    char *secondComma;
    Email email;

    /*
     * Expected:
     * EMAIL sender,subject,date
     *
     * Find commas rather than using strtok(), because the
     * subject is allowed to contain spaces.
     */
    firstComma = strchr(line, ',');
    if (firstComma == NULL)
        return;

    *firstComma = '\0';

    secondComma = strchr(firstComma + 1, ',');
    if (secondComma == NULL)
        return;

    *secondComma = '\0';

    /*
     * line is now the sender category after "EMAIL " has
     * been removed by the caller.
     */
    strncpy(email.sender, line, sizeof(email.sender) - 1);
    email.sender[sizeof(email.sender) - 1] = '\0';

    strncpy(email.subject, firstComma + 1, sizeof(email.subject) - 1);
    email.subject[sizeof(email.subject) - 1] = '\0';

    strncpy(email.date, secondComma + 1, sizeof(email.date) - 1);
    email.date[sizeof(email.date) - 1] = '\0';

    /* Remove trailing newline/carriage return from the date. */
    email.date[strcspn(email.date, "\r\n")] = '\0';

    email.priority = getPriority(email.sender);

    if (email.priority == 0) {
        fprintf(stderr, "Invalid sender category: %s\n",
            email.sender);
        return;
    }

    if (!insert(heap, email)) {
        fprintf(stderr, "Inbox is full; email was not added.\n");
    }
}

void processCommand(MaxHeap *heap, char *line)
{
    /*
     * Remove the newline first so command comparisons work
     * regardless of the input file's line endings.
     */
    line[strcspn(line, "\r\n")] = '\0';

    if (strncmp(line, "EMAIL ", 6) == 0) {
        processEmailCommand(heap, line + 6);
    }
}

```

```

    }
    else if (strcmp(line, "NEXT") == 0) {
        Email email;

        if (peekMax(heap, &email)) {
            printf("Sender: %s\n", email.sender);
            printf("Subject: %s\n", email.subject);
            printf("Date: %s\n", email.date);
        } else {
            /*
             * No email exists. The requirements only specify
             * the normal NEXT display format, so this gives a
             * clear message for the empty-queue edge case.
             */
            printf("There are no emails to read.\n");
        }
    }
    else if (strcmp(line, "READ") == 0) {
        /*
         * extractMax removes the email but we intentionally
         * do not display the extracted email.
         */
        if (!extractMax(heap, NULL)) {
            printf("There are no emails to read.\n");
        }
    }
    else if (strcmp(line, "COUNT") == 0) {
        printf("There are %d emails to read.\n", heap->size);
    }
}

/* -----
Main
----- */

int main(int argc, char *argv[])
{
    FILE *file;
    char line[MAX_LINE];
    MaxHeap heap;

    /*
     * Usage:
     * ./inbox commands.txt
     */
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <input-file>\n", argv[0]);
        return EXIT_FAILURE;
    }

    file = fopen(argv[1], "r");
    if (file == NULL) {
        perror("Could not open input file");
        return EXIT_FAILURE;
    }

    initializeHeap(&heap);

    while (fgets(line, sizeof(line), file) != NULL) {
        /*
         * Ignore completely blank lines.
         */
        if (line[0] == '\n' || line[0] == '\r' ||
            line[0] == '\0') {
            continue;
        }

        processCommand(&heap, line);
    }

    fclose(file);

    return EXIT_SUCCESS;
}

```

e. Code Analysis

Correctness

Both programs were compiled with `gcc -Wall -Wextra` (zero warnings from either) and run against the assignment's sample test file. Both produced output that matched the expected priority order (Boss newest-first, then Boss older, correct COUNT values), and both correctly handled the edge cases described in the assignment: NEXT/READ on an empty queue, two consecutive NEXTs (same email shown twice), and two consecutive READs (second email silently removed).

However, testing surfaced two real bugs. First, **both** programs omit the "Next email:" header line that the assignment's sample output requires before the Sender/Subject/Date lines - neither AI noticed this detail from the sample output format. Second, and more serious: **ChatGPT's** program declares a fixed-size array, `Email data[1000]`, inside the `MaxHeap` struct. Testing with 1,005 EMAIL commands showed ChatGPT's version silently rejects (with only a `stderr` warning) any email past the 1,000th, while Gemini's dynamically-resizing array (`malloc/realloc`, doubling capacity) correctly stored all 1,005. Since the assignment states the grader will use a different, unknown test file, this hardcoded cap is a genuine correctness risk in ChatGPT's version.

Execution Time

Both implementations use a standard binary max-heap with array-based sift-up and sift-down, giving both $O(\log n)$ insertion and extraction and $O(1)$ peek - identical asymptotic time complexity. To measure practical performance, both programs were timed (averaged over 5 runs) processing 900 EMAIL commands followed by 900 NEXT/READ pairs and a COUNT, using Python's subprocess timer. Gemini averaged about 2.4 ms and ChatGPT about 3.7 ms. This difference is small enough to be measurement noise rather than a meaningful algorithmic advantage, consistent with both using the same underlying algorithm.

Space Complexity

Gemini's `MaxHeap` stores its data in a `malloc'd` array that starts at 16 elements and doubles whenever it fills, so its memory usage is proportional to the number of emails actually stored - $O(n)$ with a small constant of overhead from occasional over-allocation after doubling. ChatGPT's `MaxHeap` embeds a fixed `Email data[1000]` array directly inside a stack-allocated struct in `main()`, reserving roughly 300 KB up front regardless of whether the inbox holds 5 emails or 995. This is not large enough to risk a stack overflow on typical systems, but it is wasteful when the inbox is small, and - as noted above - it is also a hard ceiling that cannot grow past 1,000 emails.

Maintainability

Both programs are organized into small, single-purpose functions with reasonably clear names. Gemini uses a `PriorityCategory` enum for sender categories, which is slightly more type-safe and self-documenting than ChatGPT's raw `int priority` field. ChatGPT's comments, on the other hand, more often explain *why* a design choice was made (for example, explicitly noting that `strchr` was chosen over `strtok` because subjects may contain spaces). Neither AI produced the prolog-style header block (author, dates, collaborators, inputs/outputs) or truly line-by-line comments that this assignment's rubric requires - both needed substantial additional commenting to meet the "Exceeds Expectations" tier.

f. Selection and Justification

Gemini's implementation was selected as the base for this assignment. The deciding factor was correctness under conditions the assignment explicitly warns about: the grader's test file is different from and unknown to the student, so ChatGPT's fixed 1,000-email cap is a real risk of failing the graded test file through no fault of the algorithm itself, whereas Gemini's dynamic array has no such ceiling. Gemini's use of an enum for priority categories

was also judged slightly better for long-term maintainability than raw integers. Execution time was not a deciding factor, since both were shown to be asymptotically and practically equivalent.

g. Improvements Made

Starting from Gemini's version, the following changes were made:

- **Correctness:** Added the missing "Next email:" header line before the Sender/Subject/Date output of the NEXT command, so output now matches the assignment's sample output exactly. Also added a stderr warning message when an EMAIL line is malformed (missing a field) instead of silently discarding it, making parsing problems visible during testing rather than hiding them.
- **Execution time:** Rewrote `heapify_down` from a recursive function to an iterative while-loop, removing the function-call/stack-frame overhead that was previously incurred on every level of every sift-down (called on every successful READ).
- **Space complexity:** No structural change was needed here, since Gemini's dynamic array was already the better design compared to ChatGPT's fixed-size alternative; this was confirmed rather than altered.
- **Maintainability:** Added a full prolog comment block (program name, description, inputs, outputs, author, collaborators, creation/revision dates, and a revision list) and expanded in-line comments so that nearly every line - and every function - explains what it does and why, per the rubric's requirements.

All changes were verified by recompiling with `gcc -Wall -Wextra` (zero warnings) and re-running the sample test file, an edge-case test file (empty queue, double NEXT, double READ), and a 1,005-email stress test - all of which passed with output matching expectations. The final program was also compiled and tested directly on the EECS Cycle server (`cycle1.eecs.ku.edu`), which is the environment used for grading, with output matching the assignment's sample output exactly.